

Evidence-Governed Agent Harnesses

Making Evidence Survive Long-Running Agent Execution

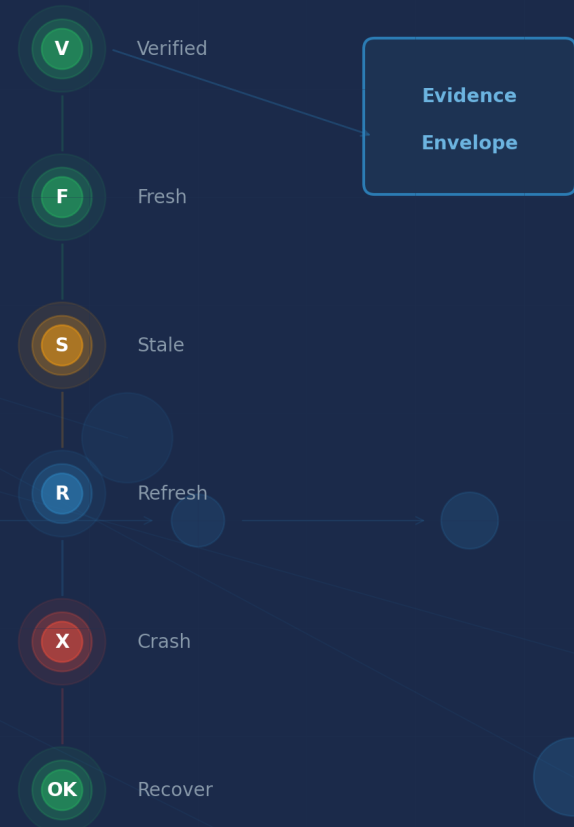
What should an agent remember after a crash at step 40 of 60?

*Not just its state — but what it verified, what it accomplished,
and what it is still allowed to do.*

Khaja Shaik

Fifth Elephant Hyderabad | 2026

EGAH



Purpose

CEGAH proposition

"Observability tells us what happened.
EGAH determines whether what happened is still sufficient
evidence for what the agent is about to do."

A working, demoable implementation why evidence must be treated as durable runtime state, not just telemetry.

- Evidence checkpoints at consequential workflow boundaries
- Crash recovery with evidence-governed continuation
- Verification history and resource accounting
- Full audit trail from request to outcome
- Interactive Traditional vs EGAH recovery comparison

Context

Agent engineering has progressed from prompts to context, harnesses, loops and graphs. Most production agent frameworks now provide checkpointing, state persistence and observability.

Critical gap: Checkpoint recovery can restore WHERE the agent was without restoring WHAT the agent knew, verified, accomplished, and is still authorized to do.

This gap becomes consequential in:

Long-running workflows

Evidence collected early may become stale before later actions execute

Crash-and-resume scenarios

Agent restores execution state but loses evidentiary context

Consequential actions

Acting on unverified or expired evidence produces unsafe outcomes

Real-World Production Scenario

Bank AI agent · Loan application · 10-step pipeline · Crash at step 7

Crashes at 2:14 PM (pod eviction) → Recovers at 5:47 PM → 3.5 hours later

What Observability Provides

- ✓ Full trace of steps 1–7
- ✓ Every LLM call, latency, token count
- ✓ Cost: ₹2.40 across 7 steps
- ✓ Pod eviction error logged
- ✓ "What happened?" — answered

What Observability Cannot Answer

- ✗ Is credit score (1:58 PM) still valid?
- ✗ Is employment verification current?
- ✗ Have fraud signals changed?
- ✗ Is risk assessment still applicable?
- ✗ Is the agent still authorized?

EGAH Recovery Plan at 5:47 PM

Step 2 — Credit Score	→ REFRESH	(stale, TTL 2h exceeded by 3h 49m)
Step 3 — Employment	→ ACT	(fresh, TTL 24h, only 3h 45m old)
Step 6 — Risk Assessment	→ REFRESH	(upstream dependency on stale credit score)

Decision: REFRESH steps 2, 6 → then RESUME at step 7

Problem Statement

10-step document analysis agent · Evidence checkpoints at steps 3, 6, 9 · Crash at step 7

Step 01	Receive Document	✓ completed
Step 02	Parse Structure	✓ completed
Step 03	Extract Entities	✓🔒 CHECKPOINT
Step 04	Classify Content	✓ completed
Step 05	Cross-Reference	✓ completed
Step 06	Assess Risk	✓🔒 CHECKPOINT
Step 07	Generate Findings	💥 CRASH
Step 08	Validate Findings	⏸ pending
Step 09	Create Report	⏸🔒 CHECKPOINT
Step 10	Finalize & Sign-off	⏸ pending

Traditional recovery resumes at step 7 — blindly. No check on entity validity, risk freshness, authorization, or evidence state.

State survived.
But did the evidence survive?

How EGAH Differs from Existing Observability

What already exists (OTel, Langfuse, Phoenix, Datadog)

- LLM generations, tool calls, retrieval
- Token usage, cost, latency
- Hierarchical traces across steps
- Quality scores and evaluations
- Answer: "What happened?"

What EGAH adds on top

- Decision-bearing evidence layer
- Evidence envelopes with provenance & TTL
- Freshness analysis and validity checks
- Policy: ACT / REFRESH / ASK / ABSTAIN
- Answer: "What can happen next?"

Dimension	Observability	EGAH
Primary question	What happened?	What can happen next?
Purpose	Records execution	Governs continuation
Nature	Retrospective	Prospective
Verification	Not built-in	First-class
Freshness	Timestamp on logs	Active validity analysis
Crash recovery	Shows trace to crash	Restores + validates evidence
Policy decisions	Not applicable	ACT / REFRESH / ASK / ABSTAIN

Solution Approach

EGAH introduces an evidence layer directly into the agent execution graph

Traditional: Plan → Tool Call → Result → Next Step

EGAH: Plan → Evidence Checkpoint → Tool Call → Result Verification → Evidence Update → Next Step

At each evidence checkpoint:

1. Captures current execution + evidence state
2. Creates an evidence envelope
(provenance, timestamp, content, status)
3. Verifies evidence against a policy
4. Decides: ACT / REFRESH / ASK / ABSTAIN
5. Persists envelope + verification durably

On crash recovery, EGAH:

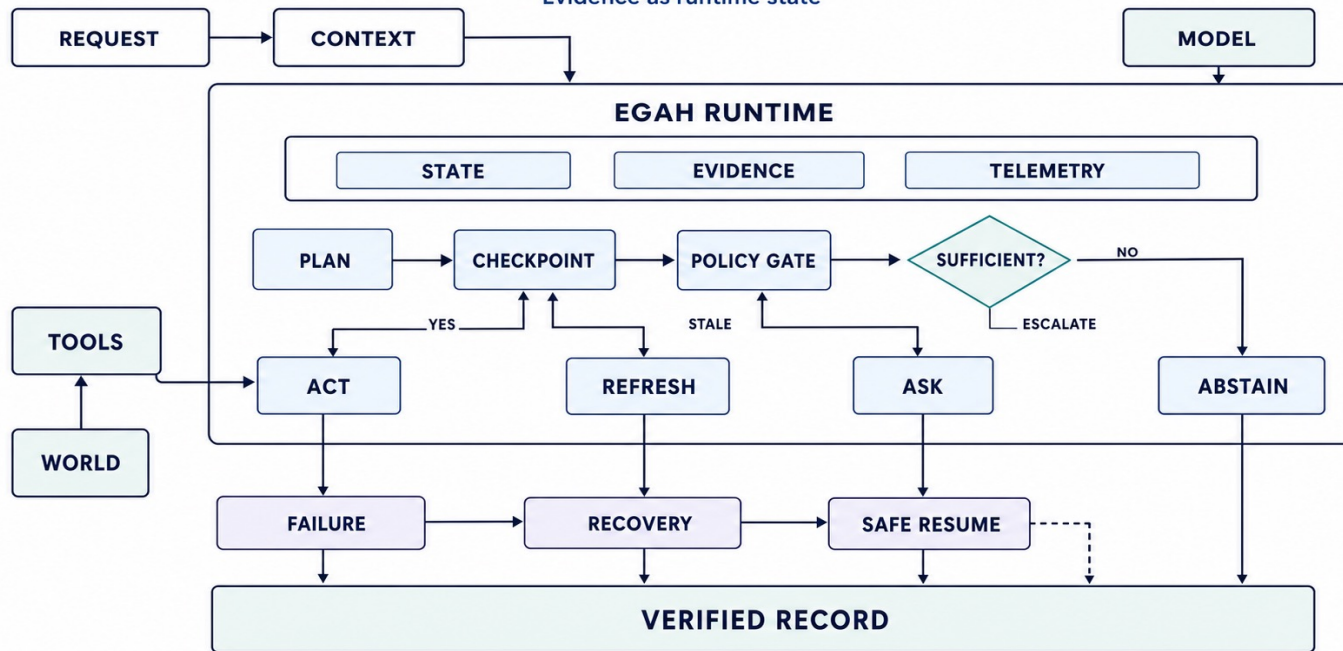
1. Restores execution state
2. Restores evidence state (all envelopes)
3. Restores verification history
4. Analyses evidence freshness & validity
5. Generates a recovery plan
(resume / refresh / escalate / abstain)

Architecture

End-to-end evidence-governed workflow

Evidence-Governed Agent Harness

Evidence as runtime state

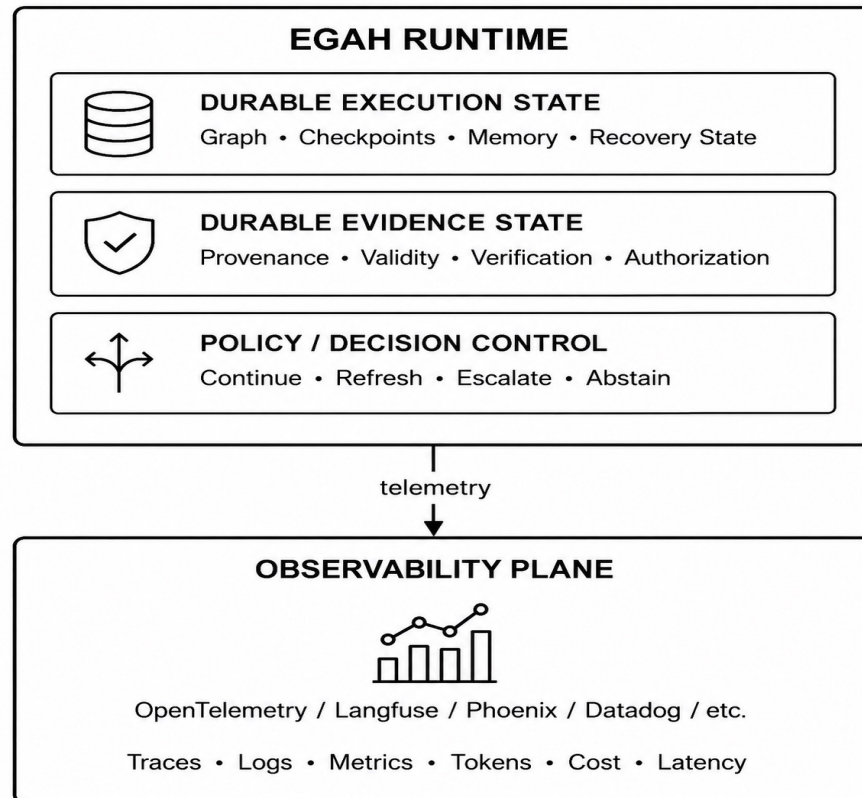


Three core subsystems: State (graph, checkpoints, memory) · Evidence (provenance, validity, verification) · Telemetry (tokens, cost, latency)

Flow: Plan → Checkpoint → Policy Gate → Sufficient? → ACT / REFRESH / ASK / ABSTAIN

Runtime & Observability Plane

EGAH sits above observability — consumes telemetry but governs continuation



Durable evidence state with verification and policy control above the observability plane

The 10-Step Example — Normal Execution

Step 01	Receive Document	✓ output recorded
Step 02	Parse Structure	✓ output recorded
Step 03	Extract Entities	✓ 📁 Evidence Envelope created + verified
Step 04	Classify Content	✓ output recorded
Step 05	Cross-Reference	✓ output recorded
Step 06	Assess Risk	✓ 📁 Evidence Envelope created + verified
Step 07	Generate Findings	✓ output recorded
Step 08	Validate Findings	✓ output recorded
Step 09	Create Report	✓ 📁 Evidence Envelope created + verified
Step 10	Finalize & Sign-off	✓ output recorded

Result: 10 steps completed, 3 evidence envelopes, 3 verifications, full audit trail

Crash at Step 7 — Traditional vs EGAH Recovery

Aspect	Traditional Recovery	EGAH Recovery
Execution state	✓ Restored	✓ Restored
Evidence state	✗ Not checked	✓ Restored (2 envelopes)
Verification history	✗ Not analysed	✓ Analysed (2 records)
Evidence freshness	✗ Unknown	✓ Step 3 stale, Step 6 fresh
Resume decision	Blind resume at step 7	Conditional: refresh then resume
Risk	May act on expired evidence	Evidence-governed continuation
Audit trail	Gap between crash/resume	Full continuity with recovery

Outcome

The PoC demonstrates in a single end-to-end flow

1

Evidence checkpoints fire at steps 3, 6, 9 — creating durable evidence envelopes with provenance, verification, freshness, and resource consumption

2

Crash at step 7 — execution state persisted, evidence state persisted, verification history persisted

3

Traditional recovery — resumes blindly with 4 warnings about unchecked evidence

4

EGAH recovery — analyses 2 envelopes, flags 1 as stale, recommends revalidation

5

Full audit trail — every step, envelope, verification, token/cost record traceable

6

Interactive comparison — side-by-side Traditional vs EGAH in Streamlit UI

Central result: A production agent harness should persist not only WHERE the agent was, but also WHAT it knew, WHAT it verified, WHAT it accomplished, and WHAT it is still authorized to do.

The Outcome Difference

Without EGAH vs With EGAH

Without EGAH	With EGAH
Resumes using 1:58 PM credit score	Detects stale score, refreshes it
Offer on potentially outdated data	Offer on current, verified data
Credit drop 782→640 = incorrect offer	Catches change, recomputes risk
Compliance gap at resume	Full evidence trail preserved
Risk: regulatory violation, financial loss	Safe, evidence-governed, auditable continuation

EGAH exists not to replace observability, but to add the layer observability was never designed to provide: evidence governance for agent continuation.

Evidence as First-Class Runtime State

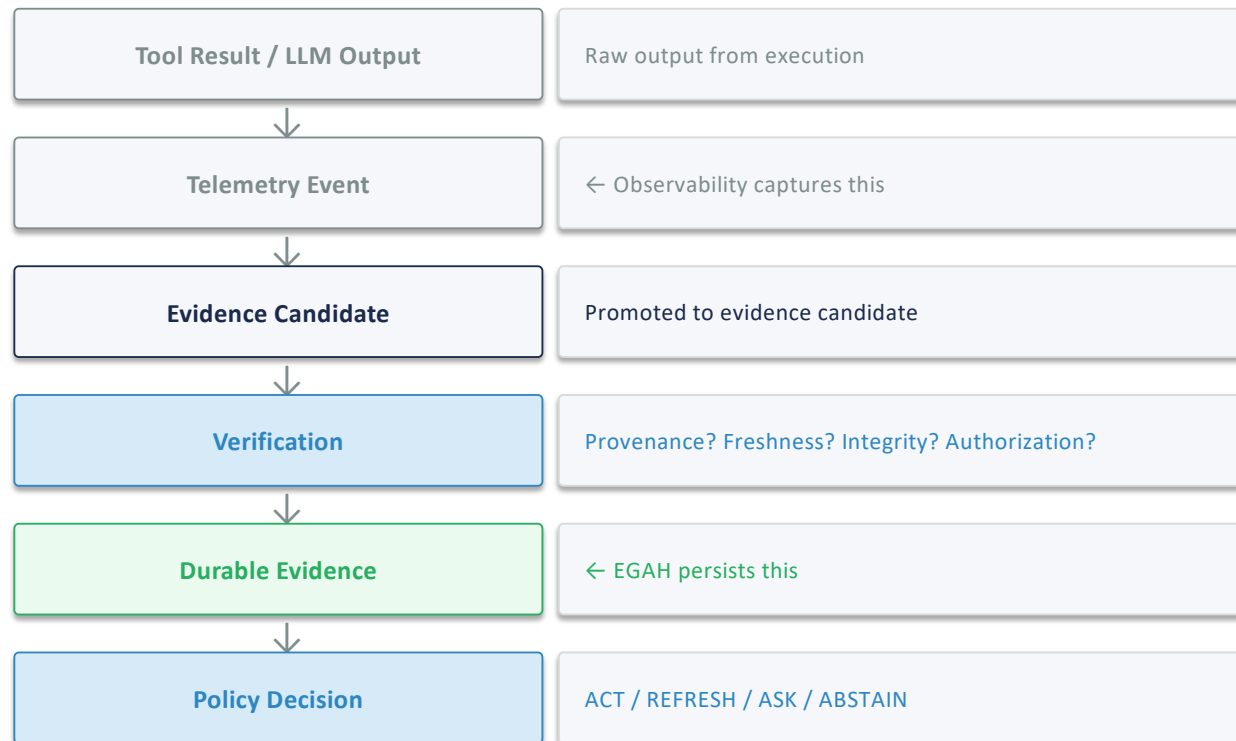
An evidence envelope carries more than a value

Identity / Provenance	Where did this evidence come from?
Observation Time	When was it captured?
Freshness TTL	How long is it considered valid?
Verification Status	Was it verified? By whom?
Content / Payload	What was actually observed?
Resource Consumption	Tokens, cost, latency for this step
Outcome Attribution	What decision did this evidence support?

From "I have a value" → "I have evidence that is traceable, verified and applicable"

The Evidence Lifecycle

Not every observation becomes evidence. Not every evidence is sufficient for action.



Key Takeaways

What practitioners should leave with

- 1 Treat evidence as first-class runtime state in long-running agents
- 2 Preserve evidence and verification history alongside execution checkpoints
- 3 Revalidate evidence at consequential boundaries — not everything, everywhere
- 4 Use evidence-backed capability boundaries for smaller models
- 5 Link resource consumption to verified outcomes, not cost alone
- 6 Measure overhead and reliability benefit before adding harness abstractions

The harness should remember not only WHERE the agent was —
but WHY it was allowed to continue.

State ≠ Evidence · Evidence needs lifecycle · Verify at boundaries · Capability boundaries · Semantic recovery · Governance earns its place